

Các nguyên lý cơ bản của LLM-powered app

Xây dựng ứng dụng và tích hợp mô hình ngôn ngữ lớn

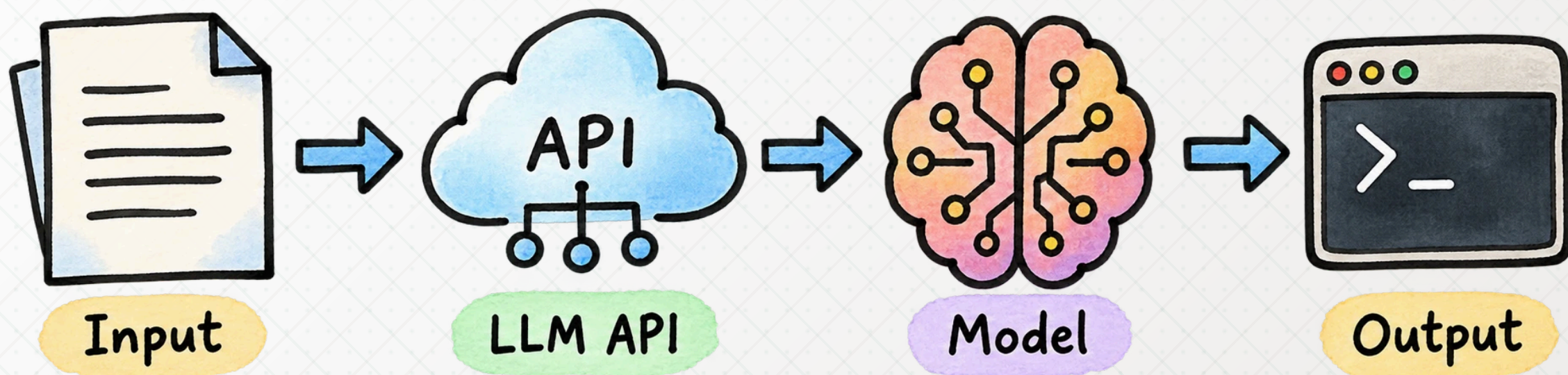
NỘI DUNG

- 01 LLM-Powered App
- 02 Prompt & Context Engineering
- 03 Structured Output
- 04 Function calling Tool Use
- 05 Bước đệm sang AI agent
- 06 Tổng kết

LLM-Powered App

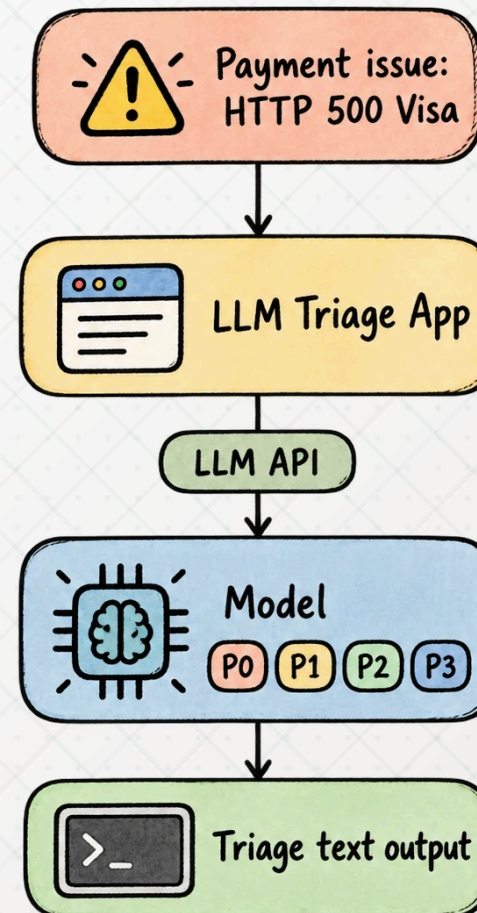
LLM-powered app

LLM-powered app là phần mềm **gọi LLM** để xử lý input và sử dụng output trong luồng xử lý của ứng dụng.



Ứng dụng Issue Triage

- ☕ Input: mô tả một lỗi trong hệ thống thanh toán.
- ☕ Ứng dụng cần phân loại issue theo mức **P0, P1, P2** hoặc **P3**.
- ☕ Trong ví dụ này, **P0 nghiêm trọng nhất, P3 ít nghiêm trọng nhất**; thực tế có thể khác giữa các tổ chức.



Output ban đầu

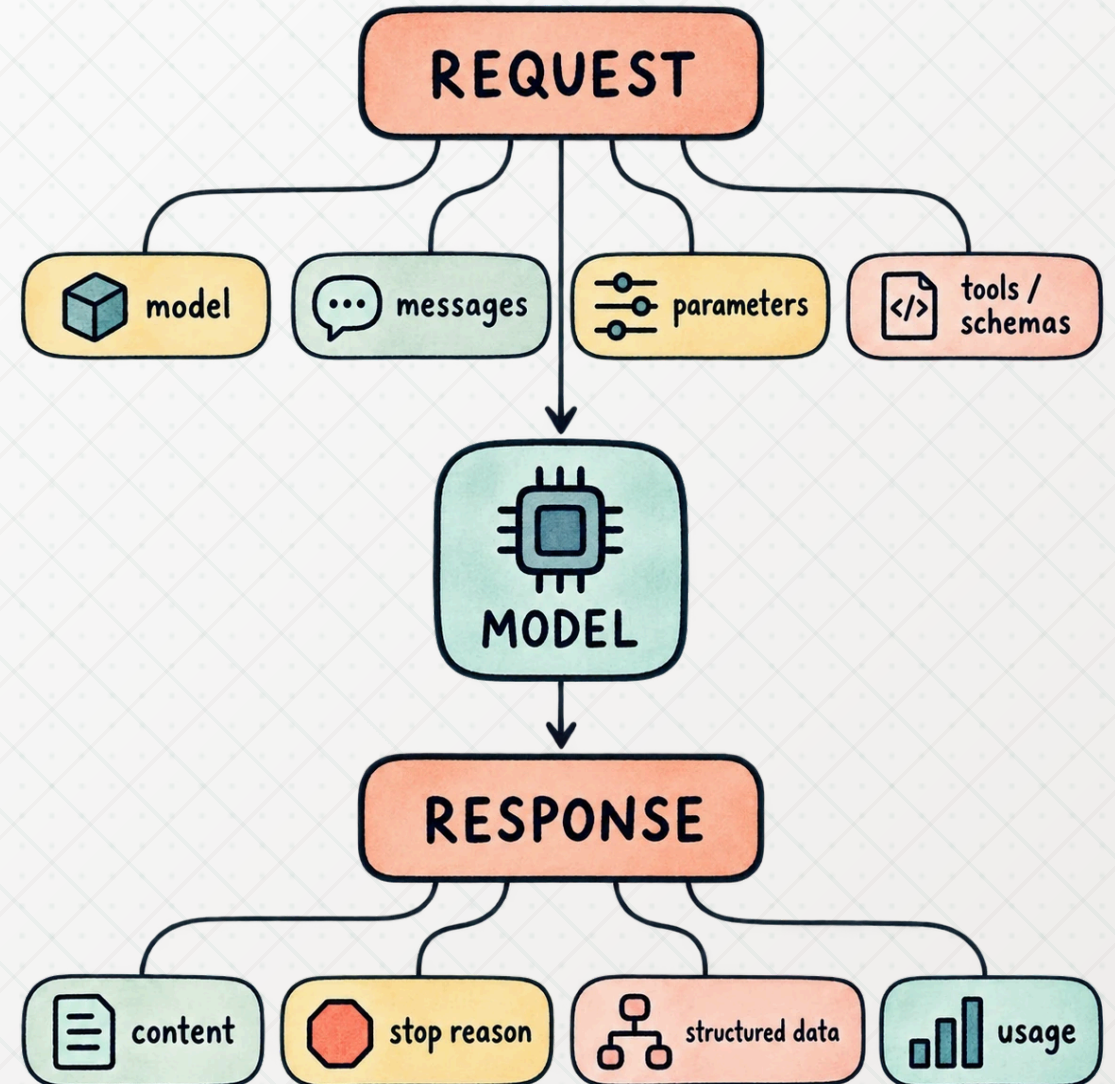
Model có thể trả:

“Đây có vẻ là sự cố nghiêm trọng ở luồng thanh toán và nên được ưu tiên xử lý ngay. Có thể xếp P0 hoặc P1 tùy phạm vi ảnh hưởng.”

- ☕ Con người có thể hiểu output này.
- ☕ Ứng dụng không thể xử lý được thông tin mơ hồ như output trên.

LLM API

- ☕ Ứng dụng gửi một **API request** có **cấu trúc** tới model.
- ☕ Request thường chứa `model`, `messages`, `generation parameters` và có thể kèm `schema` hoặc `tool`.
- ☕ Response thường chứa output, lý do kết thúc, `tool call` hoặc `structured data` nếu có, và thông tin usage.



Messages

`system` hoặc `developer`: instruction nền cho model.

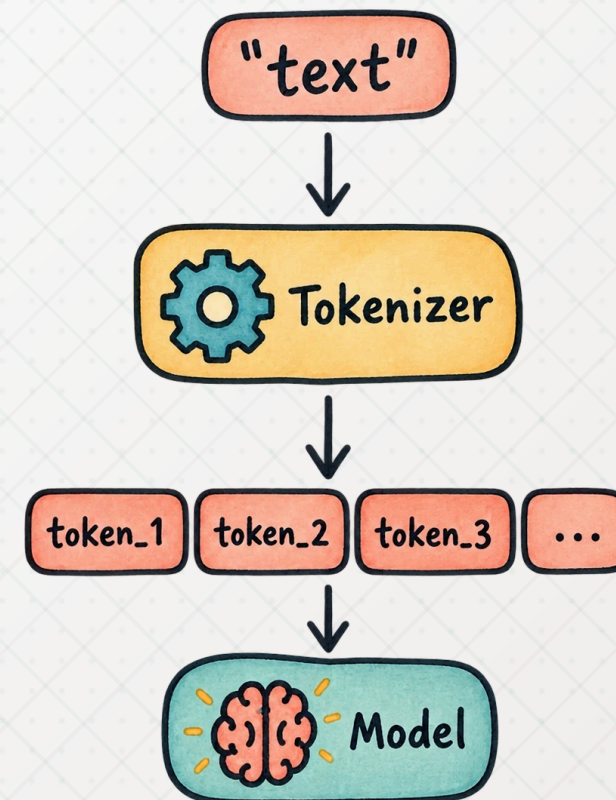
`user`: input hoặc yêu cầu từ người dùng hay ứng dụng.

`assistant`: output trước đó của model trong cùng hội thoại.

Các API có thể hỗ trợ thêm message cho `tool result` hoặc content khác.

Token

- ☕ Text được **tokenizer** chuyển thành token trước khi model xử lý.
- ☕ Một từ có thể tương ứng với một hoặc nhiều token tùy tokenizer và ngôn ngữ.
- ☕ Số token ảnh hưởng đến context window, chi phí và thời gian xử lý.



Tokenizer demo

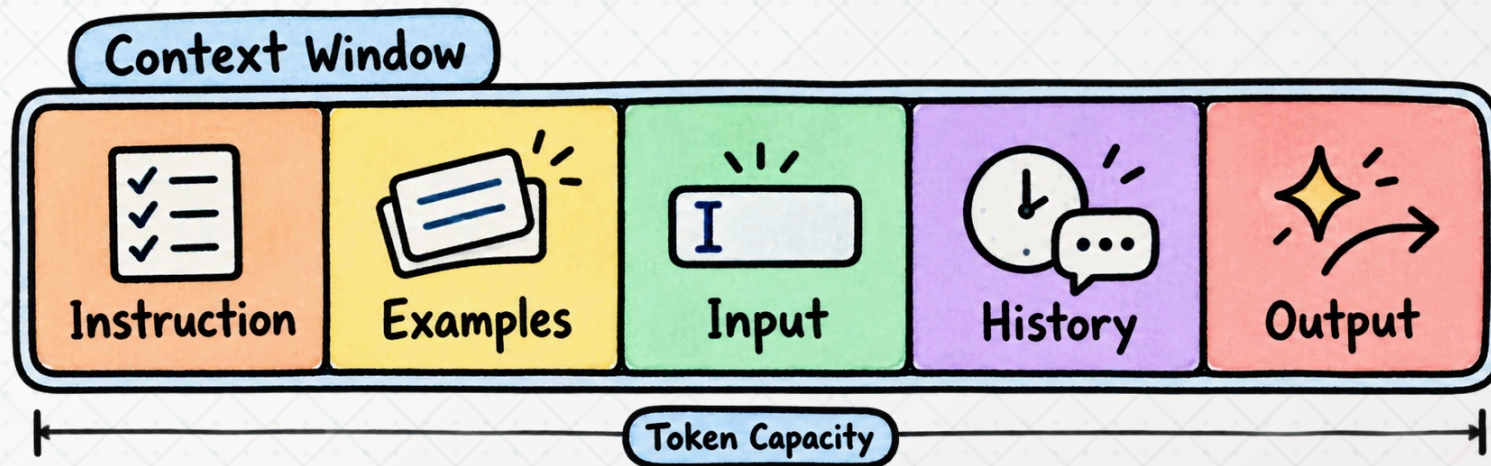
Cùng một đoạn nội dung tương đương bằng tiếng Anh và tiếng Việt:

Tokenizer	EN	VI	Tỉ lệ VI/EN
<code>cl100k_base</code>	93	228	2,45×
<code>o200k_base</code>	87	142	1,63×

- ☕ Không nên ước tính token bằng `len(text.split())`.
- ☕ Đổi **model** hoặc **tokenizer** có thể làm thay đổi chi phí dù nội dung không đổi.

Context window

- ☕ **Context window** là tổng lượng token model có thể xử lý trong một lần gọi.
- ☕ Context gồm instruction, examples, user input, history, dữ liệu bổ sung và output đang được sinh.
- ☕ Vượt giới hạn kỹ thuật có thể làm request thất bại hoặc output bị giới hạn.



Context window

Context chưa đầy vẫn có thể làm model sử dụng thông tin kém hiệu quả hơn khi có **quá nhiều nội dung** hoặc **distractor**.

1

Lost in the middle: thông tin ở giữa context window có thể được sử dụng kém hơn đầu hoặc cuối trong một số thiết lập thực nghiệm.

2

Context rot: hiệu năng có thể giảm khi input ngày càng dài và nhiều nhiễu dù vẫn còn trong giới hạn kỹ thuật.

3

Chi phí token

$$\text{cost} = (\text{input_tokens} \times \text{input_price}) + (\text{output_tokens} \times \text{output_price})$$

- ☕ Mỗi lần gọi model, phần context gửi vào lại được xử lý hoặc tính theo cơ chế cache của nhà cung cấp.
- ☕ Với workflow nhiều bước, cùng một context có thể được gửi lại nhiều lần.

Bảng giá tham khảo

Đơn vị tính: USD / 1 triệu token

Input	Cached input	Output	Model
\$5,00	\$0,50	\$25,00	<code>gpt-6-astra</code>
\$2,00	\$0,20	\$10,00	<code>gpt-5.6-sol</code>
\$1,00	\$0,10	\$6,00	<code>gpt-5.6-terra</code>
\$0,10	\$0,01	\$0,60	<code>gpt-5.6-luna</code>

Chi phí nhiều bước

Ví dụ một task 10 bước:

$$\text{step}_k \text{ input} \approx 3000 + 500 \times (k - 1)$$

- ☕ Mỗi bước có 3.000 token nền.
- ☕ Mỗi bước thêm trung bình 500 token lịch sử mới.
- ☕ Tổng input là tổng token của **10 lần gọi**, không phải token của riêng bước cuối.

Prompt caching

Một số API có thể cache phần prompt hoặc context lặp lại giữa nhiều request.

Prompt caching có thể giảm chi phí hoặc độ trễ khi prefix giống nhau được dùng nhiều lần.

Cache không làm model hiểu task tốt hơn; nó chỉ tối ưu phần xử lý lặp lại.

Generation parameters

- ☕ `temperature` và `top_p` ảnh hưởng cách model chọn token tiếp theo.
 - 🌿 `temperature` thấp, ví dụ 0–0.2: model chọn phương án có xác suất cao nhất nhiều hơn → ổn định, ít sáng tạo.
 - 🌿 `temperature` cao, ví dụ 0.8–1.2: model dễ chọn các phương án ít chắc chắn hơn → đa dạng, sáng tạo hơn, nhưng cũng dễ lệch
 - 🌿 `top_p`: giới hạn tập các token được lọt vào tập ứng viên
- ☕ `max_tokens` hoặc tham số tương đương giới hạn độ dài output.
- ☕ Không phải model hoặc API nào cũng hỗ trợ cùng một tập generation parameters.

Prompt & Context Engineering

Prompt engineering

Prompt ngắn:

Phân loại issue này: {issue}

Model chưa biết:

- ☕ dùng thang phân loại nào;
- ☕ cần giải thích hay chỉ trả nhãn;
- ☕ có được suy đoán khi thiếu dữ liệu hay không;
- ☕ P0 khác P1 thế nào.

Cấu trúc prompt

Thành phần	Vai trò	Ví dụ Issue Triage
Role/persona	Model đang đóng vai gì	Kỹ sư phân loại sự cố
Context	Hệ thống nào đang gặp vấn đề	API thanh toán đang chạy
Yêu cầu	Model cần làm gì	Phân loại mức độ nghiêm trọng
Input	Dữ liệu cần xử lý	Nội dung issue

Các thành phần bổ sung

Thành phần	Vai trò	Ví dụ Issue Triage
Ràng buộc	Điều model không được tự suy đoán	Thiếu dữ liệu phải báo thiếu
Các bước	Cách chia task khi cần	Xác định ảnh hưởng → severity
Output	Kết quả mong muốn	Severity + lý do
Ví dụ	Mẫu tham khảo	Issue đã gán nhãn đúng

Prompt có cấu trúc

- ☕ Thẻ XML giúp tách instruction khỏi dữ liệu rõ hơn.
- ☕ Prompt dài hơn không tự động làm model tốt hơn.

```
<role>
```

Bạn là kỹ sư phụ trách phân loại sự cố trên hệ thống đang chạy.

```
</role>
```

```
<task>
```

Phân loại mức độ nghiêm trọng theo P0/P1/P2/P3
Không suy đoán nếu dữ liệu chưa đủ.

```
</task>
```

```
<input>
```

```
{issue_text}
```

```
</input>
```

```
<output>
```

Trả severity và lý do ngắn gọn.

```
</output>
```

In-context learning

Kỹ thuật	Số ví dụ	Chi phí token
Zero-shot	0 ví dụ	Thấp nhất
One-shot	1 ví dụ	Thấp
Few-shot	2–5 ví dụ	Trung bình
CoT	Ví dụ + lập luận	Cao nhất

 **In-context learning:** dạy model tác vụ mới bằng context của prompt, **không thay đổi trọng số.**

 **Zero-shot:** chỉ đưa instruction và dữ liệu đầu vào.

 **One-shot / Few-shot:** thêm ví dụ mẫu để định hình cả **quy tắc** lẫn **cấu trúc output.**

Few-shot prompting

Example 1

Input: Trang chủ sai màu icon

Output: P3

Example 2

Input: Không user nào thanh toán được

Output: P0

Now classify:

{issue}

- ☕ Few-shot đưa ví dụ input-output vào context của request hiện tại.
- ☕ Trọng số model không thay đổi.

Few-shot trade-offs

Hữu ích:

- ☕ hệ thống phân loại riêng;
- ☕ format / edge case khó mô tả;
- ☕ cần minh họa boundary case.

Phản tác dụng:

- ☕ ví dụ làm context quá dài;
- ☕ ví dụ thiên lệch về một label;
- ☕ ví dụ lỗi thời;
- ☕ chỉ dùng để ép JSON trong khi đã có schema phù hợp hơn.

Chain of Thought

Zero-shot:

Prompt: Cộng các số palindrome trong c
13, 1331, 121, 73, 99, 56, 232, 7.

→ Output: 1691 (BỎ SỐT 99 - SAI)

Chain of Thought:

Prompt: Hãy làm theo từng bước:

1) Tìm tất cả các số palindrome

2) Cộng các số tìm được

→ Output: $1331+121+99+232+7 = 1790$ (ĐÚ

- ☕ **Chain of Thought (CoT):** yêu cầu model trình bày các bước lập luận trung gian trước khi cho kết quả.
- ☕ **Chia nhỏ bài toán:** hiệu quả hơn việc chỉ thêm ví dụ mẫu (one-shot / two-shot).
- ☕ **Model reasoning:** các dòng model mới tự suy luận từng bước, giảm sự phụ thuộc vào mẹo CoT thủ công.

Prompt template

```
from langchain_core.prompts import ChatPromptTemplate

triage_prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "Bạn là kỹ sư phụ trách phân loại sự cố. "
        "Phân loại issue theo P0/P1/P2/P3. "
        "Nếu thiếu dữ liệu, nói rõ thiếu dữ liệu."
    ),
    ("user", "{issue_text}"),
])
```

- ☕ Prompt có biến đầu vào có tên và có thể tái sử dụng.
- ☕ Prompt có thể được versioning, review và test cùng source code.

Prompt và context

Prompt engineering	Context engineering
Thiết kế cách mô tả task trong một model call	Quyết định thông tin nào được đưa vào context
Instruction, examples, output yêu cầu	File, history, retrieval result, tool result
Tập trung vào cách hỏi	Tập trung vào dữ liệu được nạp

Đầu ra có cấu trúc

Text output

Quay lại output đầu buổi:

“Đây là vấn đề nghiêm trọng, có thể P0 hoặc P1 tùy phạm vi ảnh hưởng.”

```
if "P0" in response:  
    notify_incident_team()
```

- ☕ Câu trả lời có thể hợp lý với người đọc nhưng không tạo ra một giá trị severity ổn định cho ứng dụng.
- ☕ `Regex` hoặc `substring` giải quyết triệt để vấn đề.

Ví dụ model trả:

“Văn bản này nên được phân loại là tech vì nó bàn về một sản phẩm công nghệ.”

Ứng dụng chỉ mong đợi:

tech

- ☕ Với người đọc, câu giải thích dài có thể hữu ích hơn.
- ☕ Với đoạn mã nguồn `if result = "tech"`, output đó không đúng interface mong đợi.

Structured output

KHÔNG:

model → prose → đoán kiểu → code

NÊN:

model → dữ liệu có cấu trúc → code

- ☕ Structured output yêu cầu model trả output theo một cấu trúc xác định trước.
- ☕ **JSON** là format phổ biến vì dễ truyền qua API và xử lý bằng mã nguồn.

IssueTriage schema

```
{
  "type": "object",
  "properties": {
    "status": {
      "type": "string",
      "enum": ["classified", "insufficient_data", "out_of_scope"]
    },
    "severity": {
      "type": ["string", "null"],
      "enum": ["P0", "P1", "P2", "P3", null]
    },
    "component": {"type": ["string", "null"]},
    "needs_urgent_response": {"type": "boolean"},
    "reason": {"type": "string"}
  },
  "required": ["status", "severity", "component", "needs_urgent_response", "reason"],
  "additionalProperties": false
}
```

JSON Schema

`enum` thu hẹp tập giá trị hợp lệ.

`required` xác định field bắt buộc.

`additionalProperties: false` từ chối field không khai báo.

Schema nên có trạng thái hợp lệ cho trường hợp **không đủ dữ liệu**.

Kiểm tra schema

Với schema yêu cầu `severity`, `component`, `needs_urgent_response` và không cho field thừa, output nào hợp lệ?

- A. `{"severity":"P0","component":"payment","needs_urgent_response":true}`
- B. `{"severity":"P0","component":"payment","needs_urgent_response":true,"note":"khẩn"}`
- C. `{"severity":"P0","component":"payment"}`
- D. `{"severity":"p0","component":"payment","needs_urgent_response":true}`
- E. `{"severity":"P0","component":"payment","needs_urgent_response":"true"}`

Pydantic

```
from typing import Literal
from pydantic import BaseModel, Field

class IssueTriage(BaseModel):
    status: Literal["classified", "insufficient_data", "out_of_scope"]
    severity: Literal["P0", "P1", "P2", "P3"] | None = None
    component: str | None = None
    needs_urgent_response: bool = False
    reason: str = Field(description="Lý do ngăn gợn dựa trên dữ liệu issue")
```

- ☕ Kiểu dữ liệu và ràng buộc nằm gần application code.
- ☕ Có runtime validation và có thể sinh JSON Schema.

Schema trong Python

Cách khai báo	Runtime validation	Khi phù hợp
Pydantic <code>BaseModel</code>	Có	Cần kiểm tra dữ liệu rõ ràng
<code>TypedDict</code>	Không	Cần type hint nhẹ
<code>dataclass</code>	Nhẹ	Cần cấu trúc Python đơn giản
JSON Schema	Tùy validator	Cần liên thông với hệ thống/ngôn ngữ khác

Prompt-only

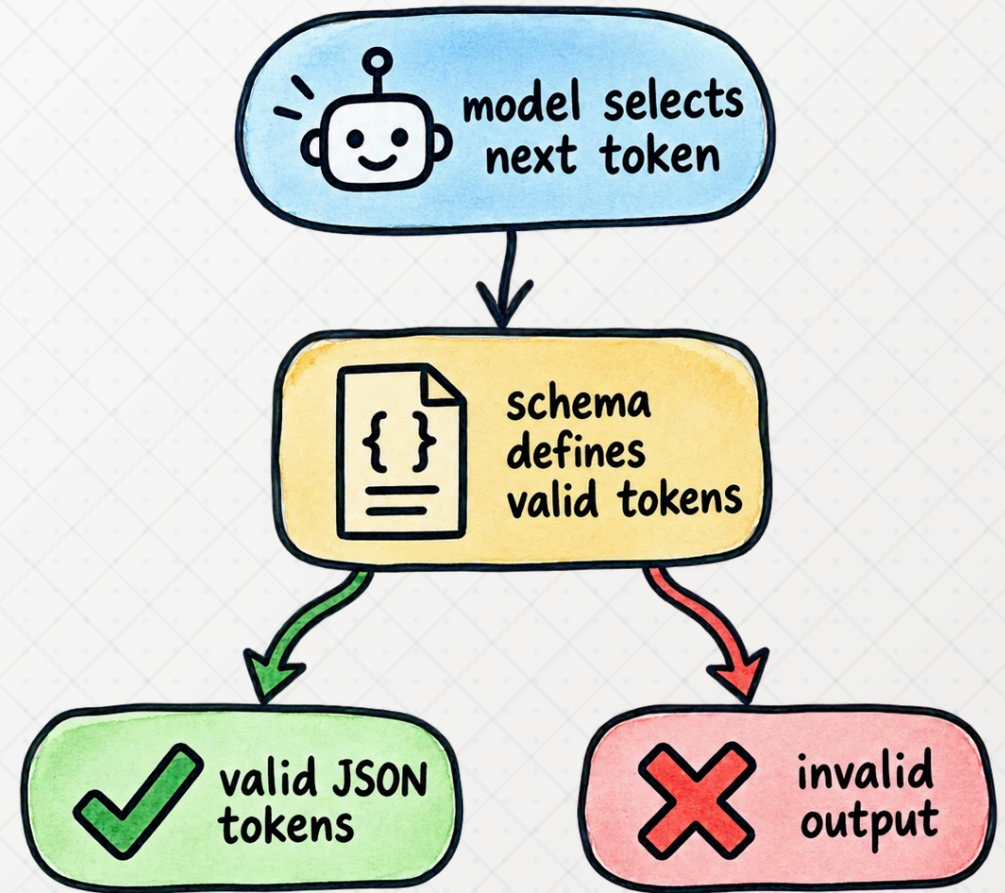
“Hãy trả JSON đúng schema này.”

Model vẫn có thể:

- ☕ thêm prose;
- ☕ bọc code fence;
- ☕ trả enum ngoài danh sách;
- ☕ thiếu field;
- ☕ trả field sai kiểu.

Constrained output

- ☕ Một số LLM API hỗ trợ structured output có ràng buộc bằng schema.
- ☕ Cơ chế này mạnh hơn việc chỉ viết “chỉ trả JSON” trong prompt.
- ☕ Tập JSON Schema được hỗ trợ có thể khác nhau giữa các nhà cung cấp và model.



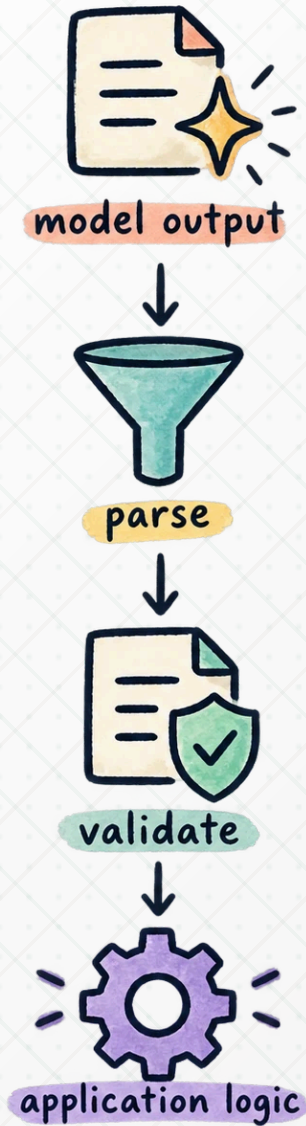
Demo B

Chạy cùng một bộ issue bằng hai cách:

1. Prompt-only
"Chỉ trả JSON theo format sau..."
2. Structured output theo schema

Quan sát:

-  output có thêm prose hay không;
-  field có thiếu hay sai type không;
-  enum có nằm trong tập giá trị hợp lệ không.



Validation

Validation có thể phát hiện:

- ☕ thiếu field;
- ☕ sai type;
- ☕ sai enum;
- ☕ dữ liệu không tuân theo schema.

Đúng schema

```
{  
  "status": "classified",  
  "severity": "P0",  
  "component": "payment",  
  "needs_urgent_response": true,  
  "reason": "Ảnh hưởng thanh toán"  
}
```

- ☕ JSON bên trái hoàn toàn **hợp lệ** theo schema.
- ☕ Severity vẫn có thể sai nếu issue thực tế chỉ là lỗi nhỏ.

Schema kiểm tra cấu trúc. Schema không kiểm tra tính đúng đắn của nội dung.

Function calling và Tool Use

Function calling

Structured output

Structured output trả dữ liệu:

```
{  
  "severity": "P0",  
  "component": "payment"  
}
```

Function calling

Function calling cho model đề xuất một hành động có cấu trúc:

```
{  
  "name": "get_component_owner",  
  "arguments": {"component": "payment"}  
}
```

Tool schema

Một tool tối thiểu cần:



`name`;



`description`;



schema của arguments.

```
from langchain.tools import tool
```

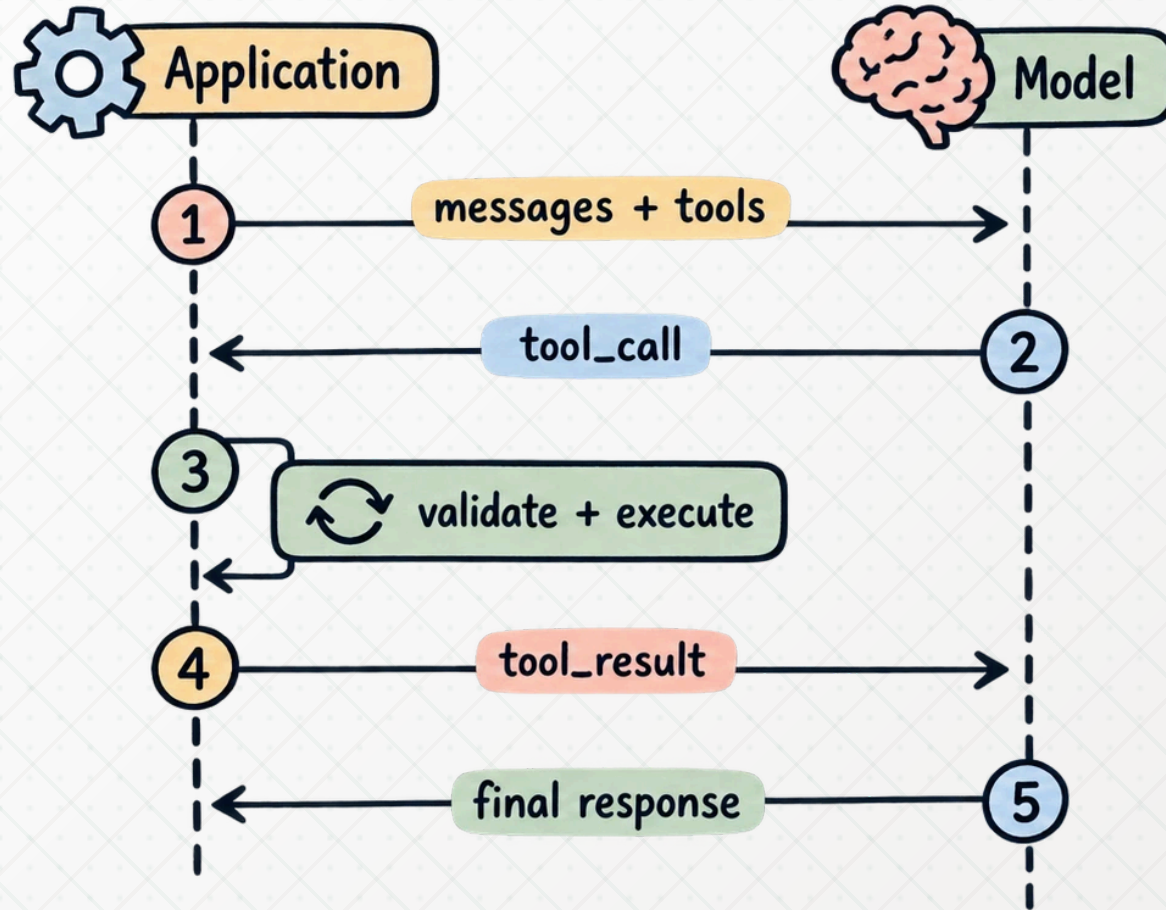
```
@tool
```

```
def get_component_owner(component: str) → str:
```

```
    """Trả về team chịu trách nhiệm cho component."""
```

```
    ...
```

Tool call



Application thực thi

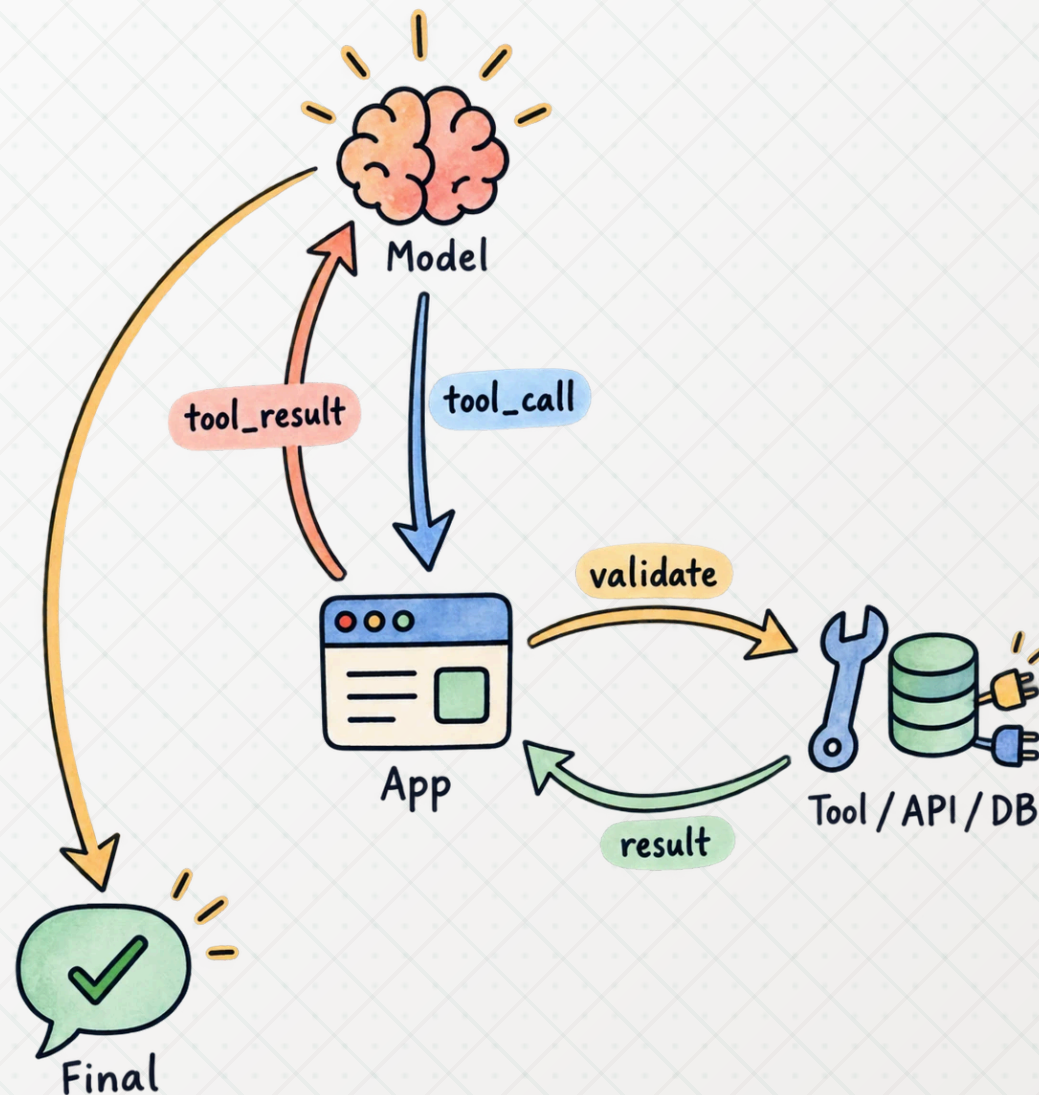
Model không thể:

- ☕ chạy Python;
- ☕ truy cập database;
- ☕ gọi REST API;
- ☕ có thêm permission chỉ vì nó sinh ra tool call.

Model chỉ sinh một object mô tả hành động mà **application** có thể thực hiện.

Tool result

- ☕ Tool result phải được gắn với tool call tương ứng.
- ☕ Model có thể dùng kết quả đó để tạo response tiếp theo.



Demo C

Input:

Nút thanh toán trả HTTP 500 với mọi thẻ Visa từ 14:30.
Hãy triage issue và cho biết team nào cần xử lý.

Model đề xuất:

```
{  
  "name": "get_component_owner",  
  "arguments": {"component": "payment"}  
}
```

Tool execution

Application thực thi:

```
get_component_owner("payment")  
→ "checkout-platform"
```

Tool result:

```
{  
  "component": "payment",  
  "owner": "checkout-platform"  
}
```

- ☕ Model nhận result rồi mới tạo final response hoặc structured triage tiếp theo.

Tool validation

Giả sử model tạo:

```
{  
  "name": "get_component_owner",  
  "arguments": {"component": "payment-api-v2"}  
}
```

- ☕ `component` đúng kiểu `string`.
- ☕ Hệ thống vẫn có thể không tồn tại component `payment-api-v2`.

Đúng kiểu không đồng nghĩa đúng ý nghĩa.

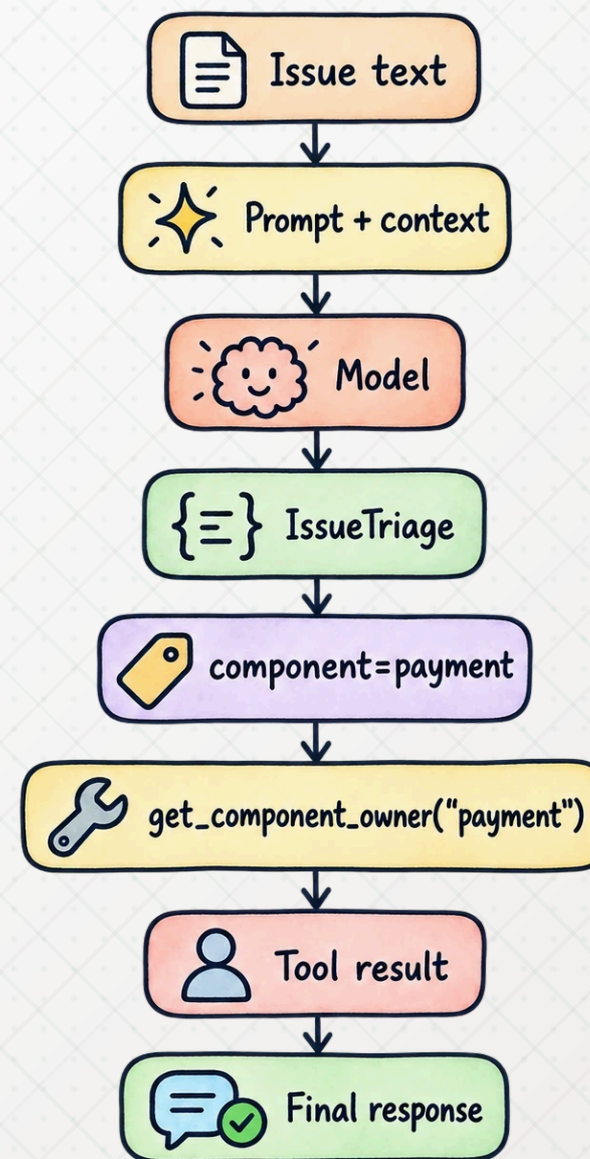
Structured output và tool call

Structured output	Tool call
Model trả dữ liệu cho ứng dụng	Model đề xuất hành động cho ứng dụng
Output tuân theo schema	Arguments tuân theo tool schema
Ứng dụng đọc dữ liệu	Ứng dụng quyết định có thực thi hay không

Cầu nối sang AI agent

Issue Triage App

- ☕ LLM-powered app giờ có thể **phân tích dữ liệu** và **đề xuất hành động**.
- ☕ Mỗi bước đều có interface mà ứng dụng có thể kiểm tra.



Chưa đạt mức agent

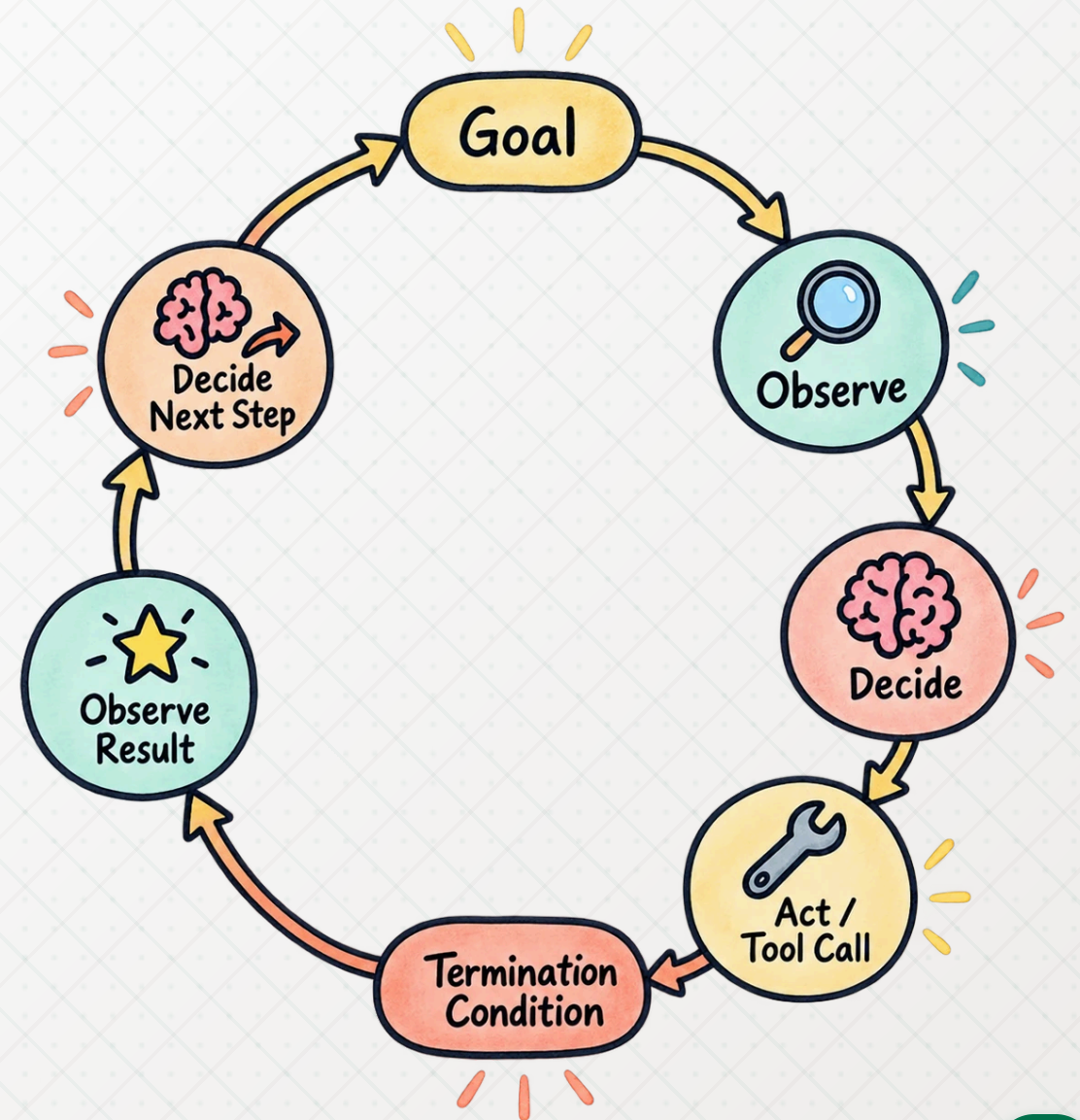
Một function call chỉ là một lần model yêu cầu một hành động có cấu trúc.

LLM-powered app hiện tại vẫn có luồng xử lý do code định trước.

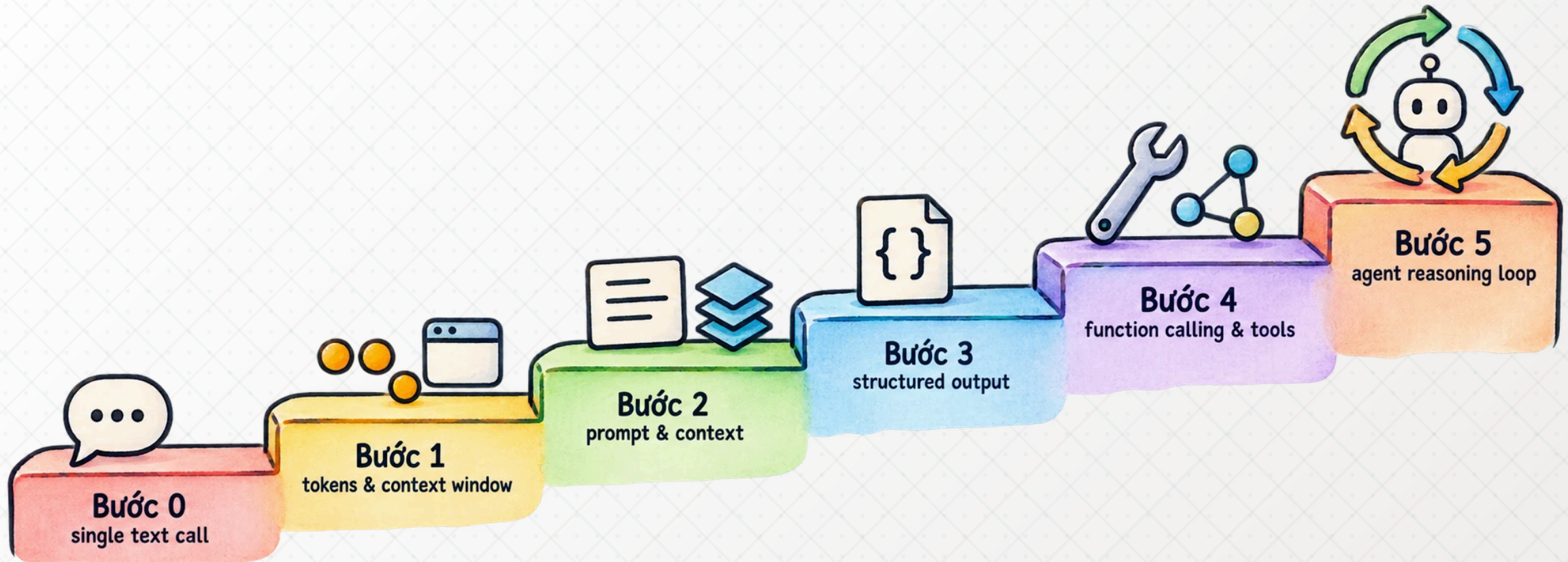
Xây dựng AI agent cần thêm vòng lặp nhiều bước và điều kiện dừng.

Agent loop

- ☕ Agent loop tổ chức nhiều lần quan sát và hành động thành một task nhiều bước.
- ☕ Buổi sau sẽ đi sâu goal, observation, reasoning/action cycle, ReAct, planning, termination condition và failure modes.



LLM-Powered App to Agent Roadmap



Ôn tập và bài tập về nhà

Quiz

LLM API và context

- ☕ Một LLM-powered app tối thiểu cần gì?
- 🌿 A. Vector database
 - 🌿 B. Agent loop
 - 🌿 C. **Application gửi input tới model và xử lý response**
 - 🌿 D. MCP

Quiz

Dạng xử lý input

☕ Model xử lý input chủ yếu dưới dạng gì?

🌿 A. Character

🌿 B. Word

🌿 C. **Token**

🌿 D. Sentence

Quiz

Context window

-  Context window nên hiểu gần nhất là gì?
-  A. Bộ nhớ lâu dài của model
 -  B. **Toàn bộ thông tin của một model call**
 -  C. Database của nhà cung cấp
 -  D. Memory giữa mọi session

Quiz

Few-shot và trọng số model

☕ Few-shot có làm thay đổi trọng số model không?

🌿 A. Có

🌿 B. **Không**

Quiz

Prose output trong automation

-  Vì sao prose output gây khó cho automation?
-  A. Prose luôn sai
 -  B. **Application phải suy ra kiểu và ý định từ text**
 -  C. JSON luôn ngắn hơn
 -  D. Model không hiểu tiếng Việt

Quiz

Vai trò của structured output

-  Structured output đảm bảo tốt nhất điều gì?
-  A. Nội dung luôn đúng sự thật
 -  B. Không hallucinate
 -  C. **Output tuân theo schema đã khai báo**
 -  D. Không cần validation

Quiz

Thực thi trong function calling

☕ Trong function calling, ai thực thi hàm?

- 🌿 A. Model
- 🌿 B. LLM provider
- 🌿 C. **Application**
- 🌿 D. JSON Schema

Quiz

Phân biệt structured output và function calling

- ☕ Structured output và function calling khác nhau ở đâu?
 - 🌿 A. Không khác
 - 🌿 B. **Structured output biểu diễn dữ liệu; function calling biểu diễn yêu cầu hành động**
 - 🌿 C. Function calling là RAG
 - 🌿 D. Structured output chỉ dùng cho frontend

BTVN#1

Xây dựng một **Issue Triage mini-app**:

- ☕ nhận mô tả issue phần mềm;
- ☕ dùng prompt template tách instruction và input;
- ☕ trả về `IssueTriage` bằng Pydantic hoặc JSON Schema;
- ☕ validate output ở phía application;
- ☕ khai báo một tool đơn giản;
- ☕ in trace `tool_call → application executes → tool_result → final response`.

Schema

IssueTriage tối thiểu gồm:

```
status:  
  classified | insufficient_data | out_of_scope  
severity:  
  P0 | P1 | P2 | P3 | null  
component  
needs_urgent_response  
reason
```

- ☕ Không parse severity bằng substring hoặc regex.
- ☕ Có **.env.example**; không commit API key thật.

Chi phí

Yêu cầu:

Tạo `docs/uoc_tinh_chi_phi.html`:

- ☕ đo token của một input mẫu;
- ☕ nêu giả định giá;
- ☕ ước tính chi phí cho 10.000 issue/tháng.

Không yêu cầu:

- ☕ agent loop, ReAct hoặc planning;
- ☕ retry/backoff và timeout policy;
- ☕ RAG;
- ☕ memory/state;
- ☕ multi-tool orchestration.

Tổng kết

- ☕ LLM-powered app bắt đầu từ một model call có cấu trúc.
- ☕ Token và context window xác định model nhận được bao nhiêu thông tin và chi phí của mỗi call.
- ☕ Prompt giúp mô tả task; context engineering quyết định thông tin nào được đưa vào model.
- ☕ Structured output biến output thành dữ liệu mà code có thể xử lý.
- ☕ Function calling biến yêu cầu hành động của model thành một tool call mà application kiểm soát.
- ☕ Các cơ chế này tạo nền tảng để tiếp tục **xây dựng AI agent** bằng agent loop ở buổi sau.

Q & A

Cảm ơn các bạn đã theo dõi!